Features, Model Search and Uncertainty

Learning a hypothesis function and testing how dependable it is

If we repeated the experiment, what would change?

From feature construction to honest comparisons, repeated runs and bootstrap reasoning.

Dania Tamayo-Vera

Three questions guide the whole experiment

01 What do we choose, and what does the algorithm learn?

02 Which observations is each step allowed to learn from?

03 What varies, and which conclusions survive that variation?

Keep returning to these questions when a workflow or a result looks convincing.

The problem: identify customers likely to leave

Predict whether a customer will leave during the next 30 days.

Information available today → h(x) a fitted tree → Prediction leave / stay

We learn a hypothesis function that approximates a relationship in the world from a finite sample.

A useful approximation can still make errors. Its performance must be evaluated.

Define success before comparing candidates

An intervention costs money, but missing customers also has a cost.

Objective: maximise mean validation precision.
Eligibility: mean validation recall must be at least 0.70.

Candidate	Mean precision	Mean recall	Eligible?
A	0.88	0.66	No
B	0.82	0.72	Yes
C	0.80	0.75	Yes

Select B under this rule. A mean-recall constraint is not an every-run guarantee.

Construct features that express a useful pattern

Every feature must be available when the prediction is made.

Raw information	Feature rule	Value
$240 spent; 6 purchases	Spend / purchases	40
2 recent purchases; 6 earlier	Recent minus earlier	-4
Plan is monthly	Is monthly plan?	1

Define the time windows and zero-purchase handling before applying the rules.

Draw the prediction-time boundary

Past transactions
Available inputs

Future cancellation
Outcome to predict

Prediction time

A cancellation date recorded next month cannot be an input to a prediction made today.

A train/test split cannot repair a feature that already reveals the future outcome.

The ledger: chosen, learned, allowed to see

Step	Chosen by us	Learned from data
Construct	Formula and time window	Nothing if the rule is fixed
Scale	Standardisation rule	Mean and standard deviation
Select	Method and k = 4	Scores and feature identities
Fit tree	Maximum depth = 4	Splits and leaf predictions

Inside a CV fold, all learned values come from that fold's training portion only.

Scaling: the rule is fixed; its numbers are learned

StandardScaler uses population standard deviation (ddof = 0).

$$z = (x - \text{training mean}) / \text{training standard deviation}$$

Training values	Fitted quantities	New validation value
100, 200, 300	Mean = 200; SD ≈ 81.65	400 becomes z ≈ 2.45

Apply the training mean and SD to validation data.
Do not recompute them using validation observations.

Scaling does not help every model in the same way

01 Distance methods: units determine which differences dominate distance.

02 Gradient descent: scaling can improve the optimisation geometry.

03 Threshold trees: positive linear rescaling preserves the possible partitions.

For a tree, spending > 200 and spending / 100 > 2 describe the same split.

Search compares whole candidate pipelines

Use 10 inputs, or a fixed selector that learns which 4 inputs to keep.

Feature option	Depth 2	Depth 4	Depth 6
All 10	Candidate A	Candidate B	Candidate C
Select 4	Candidate D	Candidate E	Candidate F

2 options × 3 depths = 6 candidate pipelines
6 candidates × 5 folds = 30 tree fits, excluding the final refit.

One fold: separate learning from evaluation

Set aside the final test set before using development data for search.

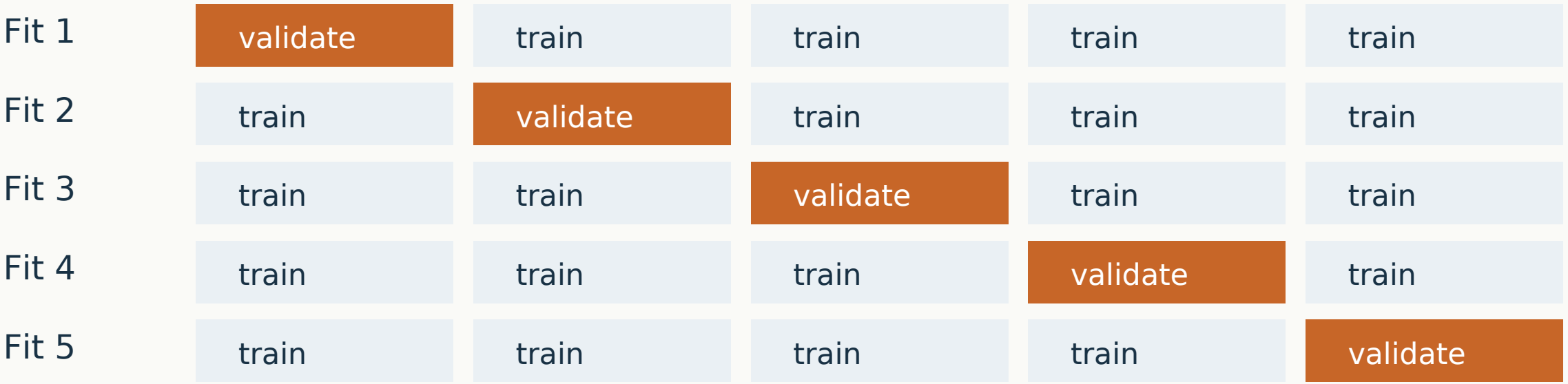


Training portion: fit the selector, then fit the tree on the selected features.

Validation portion: apply that fitted selector, then obtain predictions from that fitted tree.

Validation labels are used to score predictions, not to fit the selector or tree.

Rotate the held-out fold; refit the whole pipeline



Each candidate produces five validation precision scores and five recall scores. Average each metric separately.

Find the leak

Partner pause | 2 minutes

```
selector.fit(X_dev, y_dev)
X_selected = selector.transform(X_dev)
cross_validate(tree, X_selected, y_dev, cv=5)
```

Which observations influenced the selected features?

What must move inside each training fold?

Repair the boundary, then write the code

Leaking workflow	Repaired workflow
Select once using all development labels	For each fold, fit a new selector on training only
Validate trees on already selected inputs	Apply that selector to the validation inputs
Only the tree is refitted	Refit selector and tree in every fold

A Pipeline lets cross-validation refit the learned steps together.

Put the selector inside the cross-validation call

Runnable example: numeric, complete X_dev; binary y_dev with leave = 1.

```
from sklearn.pipeline import make_pipeline
from sklearn.feature_selection import SelectKBest, f_classif
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import StratifiedKFold, cross_validate

pipe = make_pipeline(SelectKBest(f_classif, k=4),
    DecisionTreeClassifier(max_depth=4, random_state=7))
cv = StratifiedKFold(5, shuffle=True, random_state=11)
scores = cross_validate(pipe, X_dev, y_dev, cv=cv,
    scoring={"precision": "precision", "recall": "recall"})
```

This evaluates one candidate. Compare all candidates on the same folds and apply the rule.

The winner's score is a selected score

Even a correctly cross-validated search can favour a lucky candidate.

Observed validation score
= underlying performance + sampling variation

When we choose the largest observed score, we may also be choosing a favourable fluctuation.

The winning CV score guided selection. It is not an independent estimate of the winner.

Use fresh evidence after selection

01 Search on development data using the predeclared eligibility rule.

02 Refit the selected pipeline on all development data.

03 Evaluate once on the untouched test set; report the result.

To evaluate the whole search procedure, use an outer loop that repeats search inside training.

What varies when we repeat an experiment?

Vary this	Hold this fixed	Question being asked
Split assignment	Candidates and model seed	Sensitive to the partition?
Model seed	Split and candidate	Algorithm randomness?
Search budget	Data and evaluation rule	Search coverage?
Sample or population	Learning procedure	Generalises to whom?

Different repetitions answer different questions. Name the source of variation.

Specify exactly what one run means

For the next example, fix candidates A and B and their model seeds; vary the split.

01 One run = one stratified development train/validation split.

02 Fit both complete pipelines on the same training observations.

03 Score both on the same validation observations. Repeat for five declared splits.

The final test set stays untouched. These five runs do not rerun model search.

Reproducible does not mean representative

Same data + settings + seed
can reproduce the same result.

Ten identical runs with the same seed do not explore ten different random outcomes.

Declare seeds in advance, pair the splits, and retain every result. Never pick the lucky seed.

Read the repeated runs, not just the first result

Illustrative development results | Same split for A and B within each row

Run	Precision A	Precision B	Recall A	Recall B
1	0.84	0.81	0.73	0.74
2	0.75	0.80	0.68	0.72
3	0.80	0.82	0.72	0.73
4	0.77	0.79	0.69	0.71
5	0.79	0.83	0.73	0.75

A wins on precision in run 1. Does that describe the other four runs?

Mean and SD describe different things

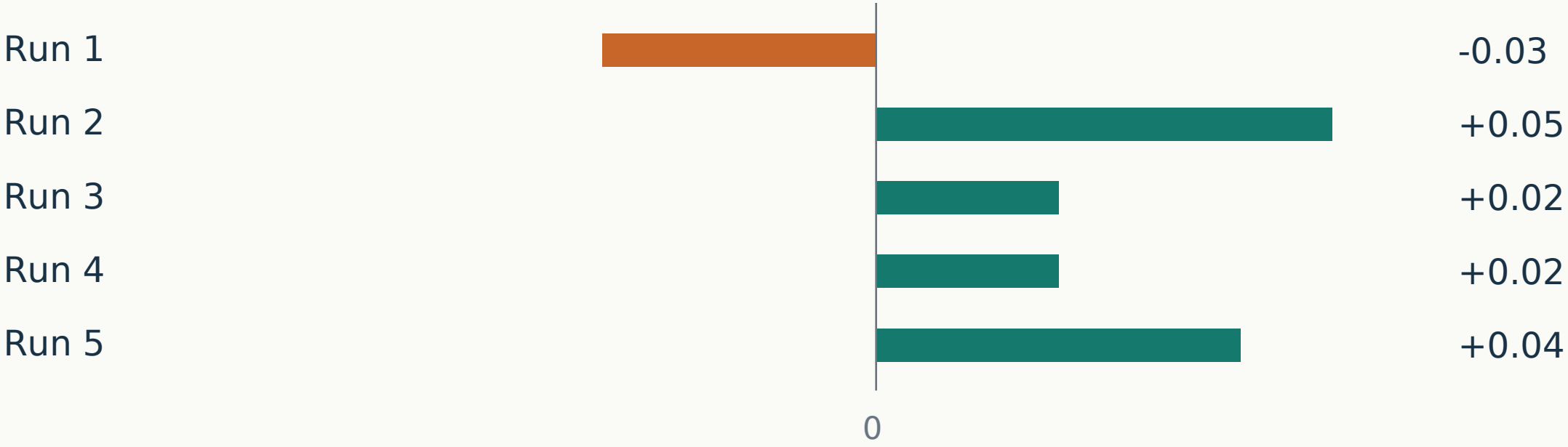
Sample SD describes dispersion across these five runs.

Metric	Candidate A	Candidate B
Mean precision	0.790	0.810
SD of precision	0.034	0.016
Mean recall	0.710	0.730

B has higher mean precision and less observed spread.
Both satisfy the stated mean-recall eligibility rule.

Paired differences preserve the comparison

Subtract within each shared split: precision B minus precision A.



Mean paired difference = +0.020; sample SD = 0.031.

Audit the fluent claim

Individual thought, then compare with a partner | 2 minutes

"B is reliably better because its precision is 0.81. Its mean recall exceeds 0.70, so it will always meet our requirement. Repeating with seed 11 proves robustness."

Underline each claim that goes beyond the evidence.
What would you need to check?

Replace confidence with a defensible statement

Across these five shared splits, B had higher mean precision (0.810 vs 0.790) and won four of five comparisons. Both met the mean-recall rule.

This supports selecting B under our rule. It does not establish performance in every future sample or population.

Overlapping splits are dependent. Run SD is not a confidence interval or proof of significance.

Bootstrap asks a different question

Keep the fitted model fixed.
How sensitive is its measured performance
to which evaluation cases we observed?

Resample whole labelled predictions with replacement,
then recalculate the metric.

Here the unit is an independent customer. Keep each true label paired with its prediction.

One bootstrap resample, by hand

Case	True	Predicted
A	leave	leave
B	stay	leave
C	leave	leave
D	stay	stay
E	leave	stay

Original recall = 2 / 3
Original precision = 2 / 3

Resample: A, A, C, D, E

Recall = 3 / 4
Precision = 3 / 3

The resample has five rows. A is repeated and B is absent; the model is never refitted.

Build a bootstrap distribution of recall

Illustrative fixed predictions on 100 independent test customers

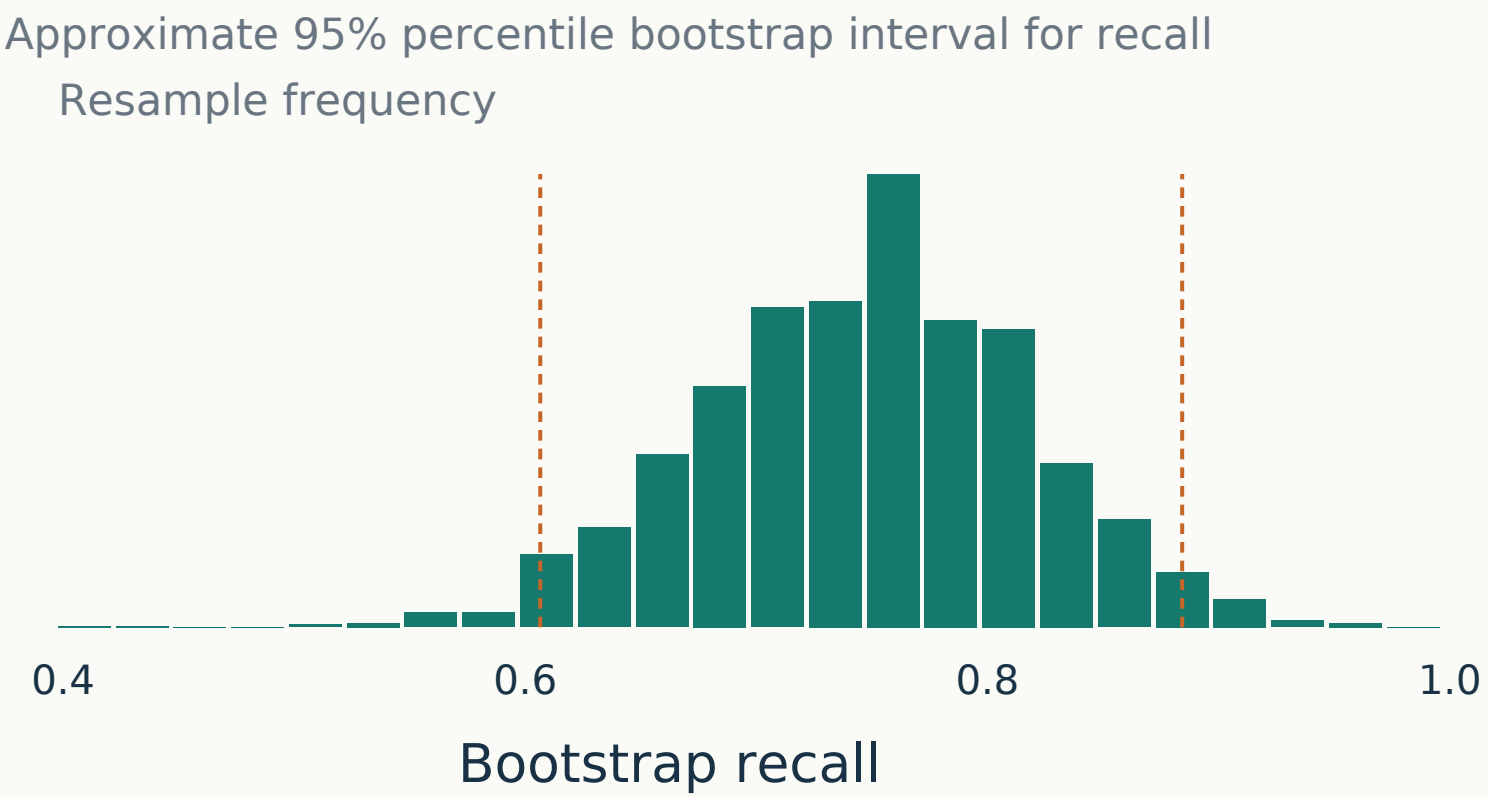
	Predicted leave	Predicted stay
Actually leave	30 true positives	10 false negatives
Actually stay	10 false positives	50 true negatives

Observed recall = 30 / (30 + 10) = 0.75

Draw 2,000 samples of 100 rows with replacement.

An interval makes the uncertainty visible

Approximate 95% percentile bootstrap interval for recall



Point estimate: 0.75

2.5th percentile: 0.61
97.5th percentile: 0.89

The interval crosses 0.70. This evidence does not establish recall above 0.70.

What this interval includes, and what it misses

01 Includes: evaluation-sample variation for this fixed model, under the sampling assumptions.

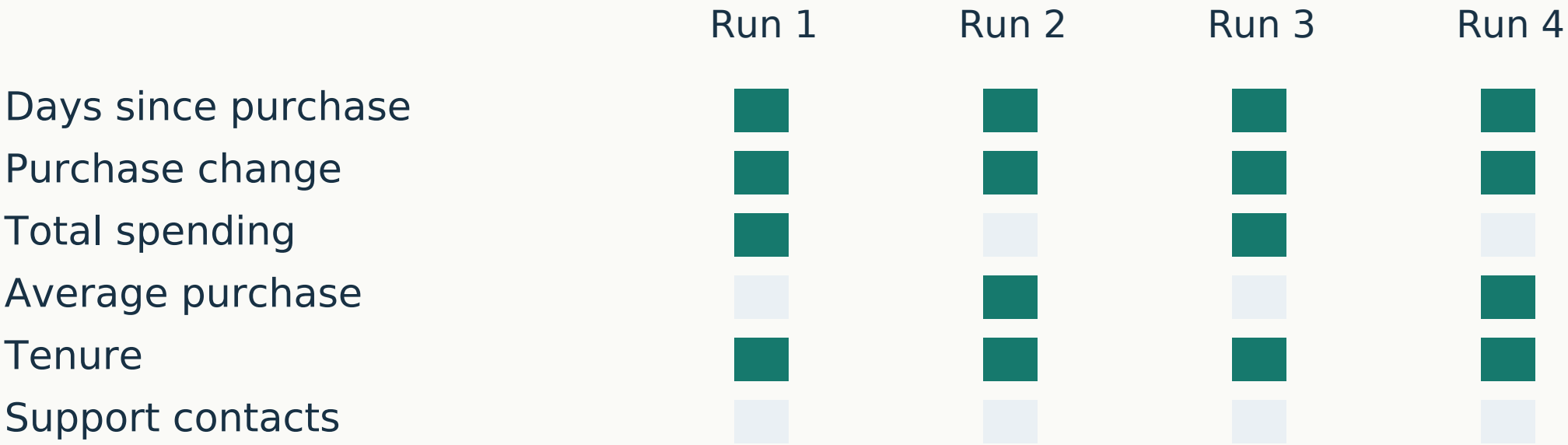
02 Misses: retraining, model search and future population changes.

03 For repeated customers or time dependence, resample appropriate groups or blocks.

Small or rare-class samples can give unstable or undefined metrics. Inspect the resamples.

Feature-selection stability is another outcome

Illustrative selections of four inputs per run | Filled cells mean selected



Correlated inputs can substitute. Next lecture: do equally accurate models explain alike?

Return to the ledger before trusting a result

Step	May learn from	May not learn from
Scale / select / fit	Current training portion	Current validation portion
Choose a candidate	Development validation results	Final test outcomes
Report final performance	Untouched test evaluation	Further tuning on that test

Features must also respect prediction time. An excluded feature is not necessarily irrelevant.

Leave an experiment log someone can reproduce

Record	Example
Question and rule	Maximise precision; mean recall ≥ 0.70
Data and pipeline	Snapshot ID, feature rules, selector, depth
What one run means	Shared split IDs; fixed model seed; full refit
Results and limitations	Every run; means, SDs, paired differences
AI partner audit	Claim, evidence checked, correction made

An AI partner can suggest checks. You must connect each conclusion to actual evidence.

Concept check: defend the learning process

Moodle activity | Counts toward the quiz grade

Work individually on six applied questions.
Allow about 8 minutes.

01 Distinguish chosen settings from learned values.

02 Track which data each step is allowed to use.

03 Match each uncertainty claim to the experiment performed.

Technical references

For follow-up reading and implementation details

scikit-learn: Common pitfalls and recommended practices
scikit-learn.org/stable/common_pitfalls.html

scikit-learn: Nested versus non-nested cross-validation
scikit-learn.org/stable/auto_examples/model_selection/plot_nested_cross_validation_iris.html

SciPy: Bootstrap and the percentile method
docs.scipy.org/doc/scipy/reference/generated/scipy.stats.bootstrap.html